2026 Spring DSAA 2011 Machine Learning

Lec 04 [demo]: from linear regression with simulated data

Zixin Zhong
Hong Kong University of Science and Technology (Guangzhou)

In [1]:
```
##  basic function
import numpy as np, pandas as pd

## import function for pLot
import matplotlib.pyplot as pit

## import function for Logistic regression
from sklearn.linear_model import LogisticRegression
```

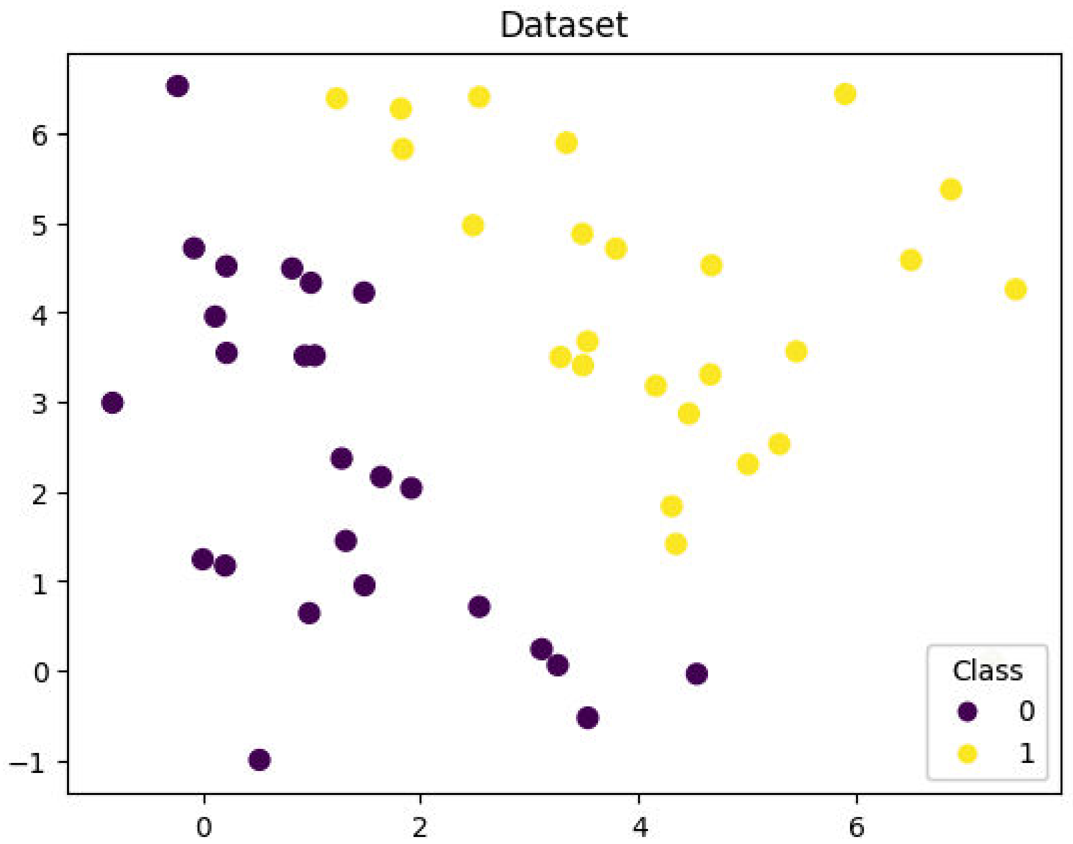
In [2]:
```
#### randomLy generate dataset
n_train = 50;
n_test = 2;

np.random.seed(0)
X_train_raw = np.random.rand(n_train,2) + np.random.normal(loc=2, scale = 2, size=(n_train,2) )  ;

X_sum = X_train_raw[:,0]*1.5+X_train_raw[:,1]; #np.sum(X_train_raWjaxis=l);
X_sum_bar = np.median(X_sum)
# ff(x) if condition eLse g(x) for x in sequence]
y_train=np.array([ 0 if tmp < X_sum_bar else 1 for tmp in X_sum ])
X_train = X_train_raw;
X_train[:,0] = X_train[:,0] + [ 0.5 if tmp >0 else 0 for tmp in y_train ]

print('labels in the dataset:')
print(y_train)
print(' ')
print('Feature vectors in the dataset:')
np.set_printoptions(precision=5,suppress=True)
print(X_train)
```

```
labels in the dataset:
[0 0 1 1 0 1 1 1 1 1 1 0 0 0 1 1 1 0 1 1 0 1 0 1 1 0 0 0 1 0 0 1 1 0 0 1 0
 0 0 0 1 1 0 0 1 1 1 0 0 0]

Feature vectors in the dataset:
[[ 0.21851  4.51684]
 [ 3.53409 -0.5276 ]
 [ 5.90016  6.43767]
 [ 5.29515  2.53192]
 [ 0.82216  4.49234]
 [ 2.48537  4.97379]
 [ 3.48459  4.87887]
 [ 3.28377  3.50028]
 [ 2.54122  6.40436]
 [ 3.53198  3.67399]
 [ 7.24492  0.10364]
 [-0.07949  4.71932]
 [-0.22797  6.52716]
 [ 1.31612  1.44976]
 [ 6.86773  5.37569]
 [ 6.49967  4.58632]
 [ 1.2337   6.38856]
 [ 1.48278  4.22255]
 [ 5.0066   2.30691]
 [ 4.67191  4.52623]
 [ 3.11236  0.23823]
 [ 3.79411  4.713  ]
 [ 1.27763  2.37137]
 [ 1.84008  5.82745]
 [ 4.16002  3.17863]
 [ 1.03036  3.5171 ]
 [ 1.63971  2.16571]
 [ 0.93718  3.51418]
 [ 4.30629  1.83669]
 [ 3.25832  0.0583 ]
 [-0.82355  2.98916]
 [ 3.48968  3.40825]
 [ 7.46287  4.25768]
 [ 0.99535  4.33113]
 [ 0.20613  1.17293]
 [ 3.33998  5.89534]
```

```
[ 1.48725  0.95197]
[ 2.54236  0.71223]
[ 4.53608 -0.03967]
[ 0.0012   1.24309]
[ 1.82192  6.27333]
[ 4.46299  2.86757]
[ 0.11573  3.95412]
[ 0.52282 -0.9956 ]
[ 5.45201  3.56318]
[ 4.66029  3.30487]
[ 4.34546  1.41428]
[ 0.22092  3.54638]
[ 0.97969  0.64101]
[ 1.91788  2.03965]]
```

In [3]:

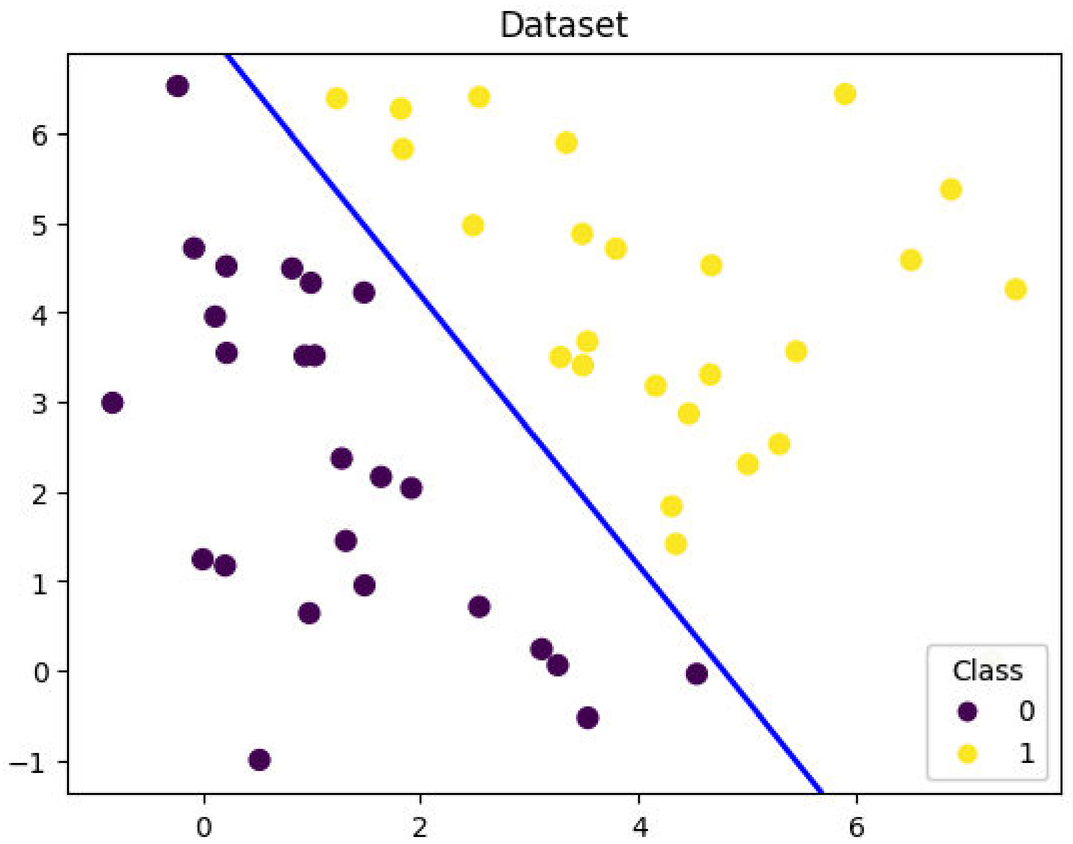
```
## Visualize the dataset
fig, ax = plt.subplots()

scatter = ax.scatter(X_train[:,0],X_train[:,1], c=y_train, s=50)#, cmap='viridis'
# produce a legend with the unique colors from the scatter
legend1 = ax.legend(*scatter.legend_elements(),
                    loc="lower right", title="Class")
ax.add_artist(legend1)

plt.title('Dataset')
plt.show()
```



In [4]:
```
#### load the logistic regression model
lr_clf = LogisticRegression()

#### fit the model with dataset
lr_clf = lr_clf.fit(X_train, y_train) # y=theta0+theta1*x1+theta2*x2

## observe theta
print('the weight of Logistic Regression (\u03B8):       ',lr_clf.coef_)

## observe theta_0
print('the intercept of Logistic Regression (\u03B8_0):  ',lr_clf.intercept_)
```

```
the weight of Logistic Regression (θ):        [[2.00899 1.33119]]
the intercept of Logistic Regression (θ_0):   [-9.60226]
```

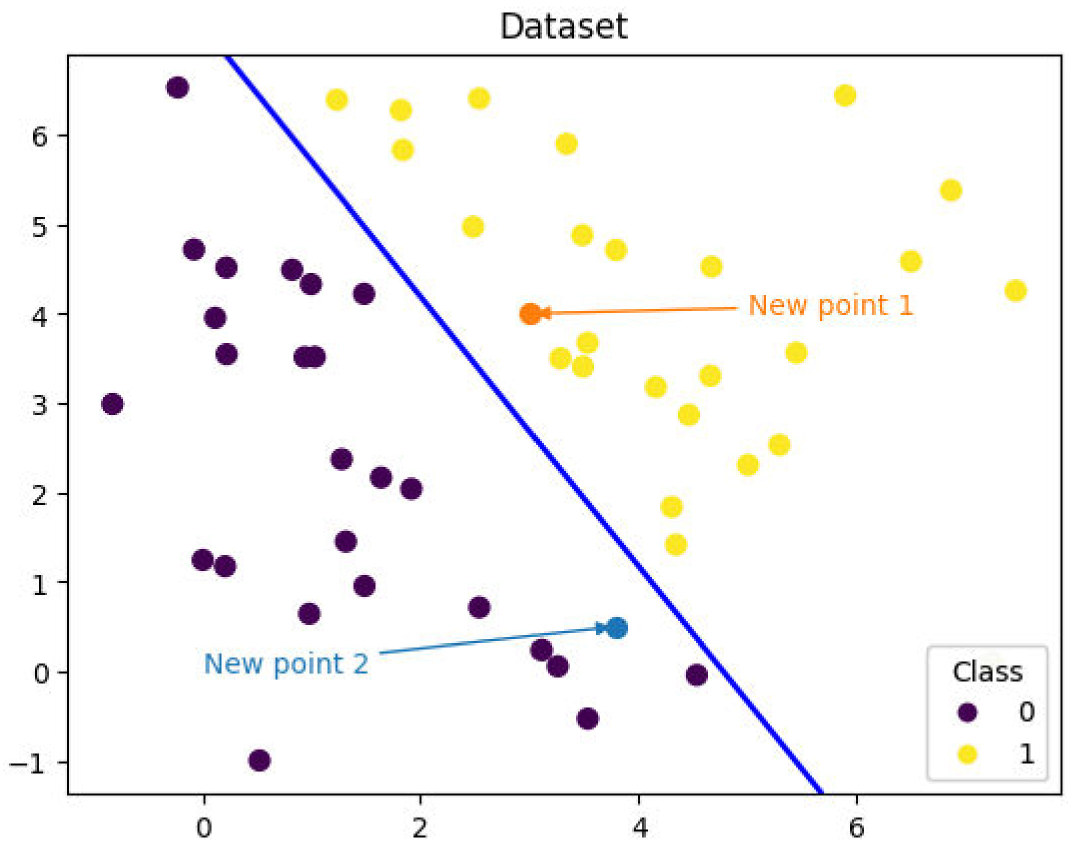
In [5]:
```python
## Visualize the boundary
fig, ax = plt.subplots()

scatter = ax.scatter(X_train[:,0],X_train[:,1], c=y_train, s=50)#, cmap='viridis'
# produce a legend with the unique colors from the scatter
legend1 = ax.legend(*scatter.legend_elements(),
                    loc="lower right", title="Class")
ax.add_artist(legend1)

nx, ny = 200, 100
x_min, x_max = ax.get_xlim()
y_min, y_max = ax.get_ylim()
x_grid, y_grid = np.meshgrid(np.linspace(x_min, x_max, nx),np.linspace(y_min, y_max, ny))

z_proba = lr_clf.predict_proba(np.c_[x_grid.ravel(), y_grid.ravel()])
z_proba = z_proba[:, 1].reshape(x_grid.shape)
ax.contour(x_grid, y_grid, z_proba, [0.5], linewidths=2., colors='blue')

plt.title('Dataset')
plt.show()
```



In [6]:

```
## Visualize new data points
fig, ax = plt.subplots()

## new point 1
X_test_1 = np.array([[3, 4]])
ax.scatter(X_test_1[:,0],X_test_1[:,1], s=50,c ='tab:orange')#, cmap='viridis')
ax.annotate(text='New point 1',xy=(3, 4),xytext=(5,4),color='tab:orange',arrowprops=dict(arrowstyle='-|>',connectionstyle='arc

## new point 2
X_test_2 = np.array([[3.8, 0.5]])
ax.scatter(X_test_2[:,0],X_test_2[:,1], s=50, c='tab:blue')#, cmap='viridis')
ax.annotate(text='New point 2',xy=(3.8, 0.5),xytext=(0, 0),color='tab:blue',arrowprops=dict(arrowstyle='-|>',connectionstyle='
```

```
############ dataset
scatter = ax.scatter(X_train[:,0],X_train[:,1], c=y_train, s=50)#, cmap='viridis'
# produce a legend with the unique colors from the scatter
legend1 = ax.legend(*scatter.legend_elements(),
                    loc="lower right", title="Class")
ax.add_artist(legend1)

############ boundary
ax.contour(x_grid, y_grid, z_proba, [0.5], linewidths=2., colors='blue')

plt.title('Dataset')
plt.show()
```



In [7]:

```python
## Apply the model for prediction in new points and the dataset
X_test_1_predict = lr_clf.predict(X_test_1)
X_test_2_predict = lr_clf.predict(X_test_2)

print('The New point 1 predict class:\n',X_test_1_predict)
print('The New point 2 predict class:\n',X_test_2_predict)

## 由于逻辑回归模型是概率预测模型（前文介绍的 p = p(y=1|x,\theta)）,所以我们可以利用 predict_proba 函数预测其概率
y_test_1_predict_proba = lr_clf.predict_proba(X_test_1)
y_test_2_predict_proba = lr_clf.predict_proba(X_test_2)

print('The New point 1 predict Probability of each class:\n',y_test_1_predict_proba)
print('The New point 2 predict Probability of each class:\n',y_test_2_predict_proba)
```

```
The New point 1 predict class:
 [1]
The New point 2 predict class:
 [0]
The New point 1 predict Probability of each class:
 [[0.14811 0.85189]]
The New point 2 predict Probability of each class:
 [[0.78626 0.21374]]
```